Ain Shams University, Cairo, Egypt
Faculty of Engineering,
International Credit Hours Engineering Programs (ICHEP)

Desktop Hotel Reservation System

(Milestone-1 Report)

Course Name: -
CSE241 Object-Oriented Computer Programming

Submitted to: -
Dr. Mahmoud Khalil
Eng. Mario

GitHub Repository: -
https://github.com/YassinMohamed07/ProjectOOPHotelSystem

Prepared by: -
Abdullah Alaa Abdelkader (25P0139)
Mohamed Alaaeldin Elsharkawy (25P0202)
Karim Ismail Samy Khalil (25P0060)
Seif Hazem Mahmoud (25P0005)
Yassin Mohamed Yehia (25P0006)

Date of submission: -
26 April 2026

1.0 Problem Description

The objective of this project is to design and implement the backend foundations of a Desktop Hotel Reservation System using Java. The current problem requires building a robust architecture that handles guest registrations, staff operations, room management, and booking algorithms using core Object-Oriented Programming (OOP) principles.

2.0 Team Responsibilities & Contributions

Work was divided fairly among the 5 team members:

- **Mohamed:** Designed the core system interfaces (**Payable**, **Manageable**). Modeled the Staff parent class and Admin subclass, implementing full CRUD operations for rooms, amenities, and room types. Authored the final UML Class Diagram.
- **Abdallah: Built** the foundational skeleton classes (**Room Type, Amenity, and Room**) and the Database class using static **Array List<T>** fields. Hardcoded the dummy data for immediate testing and led the Quality Assurance (QA) testing.
- **Karim:** Modeled the Guest class and implemented secure registration and login logic. Created the room search/filtering engine and engineered system-wide input validation using custom Java Exceptions.
- **Seif:** Designed the **Reservation** class and built the core algorithmic logic to make, view, and cancel reservations. Implemented complex date verification to prevent overlapping dates and double booking.
- **Yassin**: Modeled the **Receptionist** and **Invoice** classes. Implemented check-in/check-out functions and developed the financial logic by calculating room totals and processing payments via the **Payable** interface.

3.0 Detailed Solution & Techniques Used

3.1 Object-Oriented Principles:

The system is designed using core Object-Oriented Programming (OOP) principles to make the code organized, easy to maintain, and simple to expand. We used the following concepts:

- **Encapsulation:** We kept all data (like guest balances or room prices) private. To access or change this data, the system uses specific "getters" and "setters" that check for errors first for example, preventing a user from entering a negative balance.
- **Inheritance:** To avoid writing the same code twice, we created a general Staff parent class. Both Admin and Receptionist inherit basic info from it, like their username and birth date, while having their own unique jobs.
- **Abstraction and Interfaces:** We used "contracts" called interfaces, like Payable and Manageable, to set strict rules for how the system handles payments and management tasks.
- **Polymorphism:** This allows the system to treat different objects (like an Admin or a Receptionist) as a general "Staff" member when needed, making the code much more flexible.

3.2 Data Management & Validation

- **In-Memory Database:** We built a Database class that stores all our lists of guests, rooms, and bookings in one place while the program is running.
- **Pre-population:** We added "dummy data" (fake guests and rooms) into the system right away so it can be tested as soon as it starts.
- **Custom Exception Handling:** Instead of the program just crashing when something goes wrong, we created custom error messages. For example, if a user tries to book a room that isn't free, the system throws a RoomNotAvailableException.
- **Type Safety via Enums:** We used "Enums" for fixed categories like Gender (MALE/FEMALE) or Roles (ADMIN/RECEPTIONIST). This prevents types and ensures only valid options can be selected.

4.0 Sample Output & Test Cases

This section confirms the system's stability by testing core logic through the console.

4.1 Successful Scenarios

- **Registration & Login:** Verified that **Guest** new objects are successfully created, stored in the **Database list**, and accessible via secure login.

```
/Users/y/Library/Java/JavaVirtualMachines/openjdk-26/Contents/Home/bin/java ...
--- Welcome to the Desktop Hotel Reservation System ---

1. Login as Guest
2. Login as Staff
3. Register New Guest
4. Exit
Select an option (1-4): 1
Enter Username:
Abdullah
Enter Password:
Abdalaa.2007
Login successful: Abdullah

--- Guest Menu ---
1. Search & Book a Room
2. View My Reservations
3. Cancel a Reservation
4. Checkout & Pay
5. Exit
Select an option (1-5):
2
---My reservations---
1 Room number: 102   checkin date: 2026-04-25    checkout date: 2026-04-27   Reservation status: PENDING
```

- **Room Search:** Confirmed the search engine correctly filters the room list based on specific RoomType and **Amenity** criteria.

- **Booking Process:** Validated that a **Reservation** is successfully generated and linked to an **Invoice** when dates are available.

```
--- Guest Menu ---
1. Search & Book a Room
2. View My Reservations
3. Cancel a Reservation
4. Checkout & Pay
5. Exit
Select an option (1-5):
1
Enter wanted checkin date (e.g., 2026-04-26):
2026-04-26
Enter wanted checkout date:
2026-05-01
Enter Room type (Single,Double,Suite): single
Enter your maximum budget per one night:
23000

--- Available Rooms found ---
1. Room number: 202 Room Type: Type name: Single basePrice: 2000.0 Amenities in the room: [ name: Basic
 Wi-Fi its price: 0.0,  name: Standard Smart TV its price: 0.0,  name: Air Conditioning & Heating its price:
 0.0,  name: In-Room Safe its price: 0.0,  name: Daily Housekeeping its price: 0.0,  name: Work Desk & Chair
 its price: 0.0,  name: Coffee & Tea Station its price: 0.0,  name: Premium Bathrobes its price: 0.0,  name:
 Complimentary Mini-Bar its price: 0.0,  name: Separate Lounge Area its price: 0.0]

Which room do you want to book: 1
Reservation created successfully for room 202
Room reserved successfully!
```

- **Receptionist (Front Desk) controls:** manage checkin/checkout

```
--- FRONT DESK DASHBOARD ---
1. View All Guests
2. View All Rooms
3. View All Reservations
4. View All Invoices
5. Check-In a Guest
6. Check-Out a Guest & Process Payment
7. Update Reservation Status
8. Finalize Expired Reservations
9. Logout
Select an option (1-9): 5

Enter Guest Username to Check-In: Karim

--- Active Reservations for Karim ---
1. Room #202 | 2026-06-15 to 2026-06-17 | Status: PENDING
2. Room #202 | 2026-04-26 to 2026-04-29 | Status: PENDING

Select reservation number to check in (0 to cancel): 2
== CHECK-IN SUCCESSFUL ==
Guest: Karim
Room: #202
Check-in Date: 2026-04-26
Check-out Date: 2026-04-29
```

- **Admin Controls:** Confirmed that **Admin** users can update room prices and manage the hotel inventory in real time.

```
Log in successful: Seif

--- ADMIN DASHBOARD ---
1. View Data
2. Add Data
3. Update Data
4. Delete Data
5. Register new staff
6. Logout
Select an option (1-6): 3

--- What would you like to UPDATE? ---
1. Update a Room
2. Update a Room Type
3. Update an Amenity
Select an option (1-3): 2

--- Update Room Type Price ---
Enter the Name of the Room Type to update (e.g., Suite): single
Found 'Single'. Current Price: $2000.0
Enter NEW Base Price: $3000
Success: 'Single' price updated to $3000.0
```

4.2 Error Handling & Bug Prevention

- **Double-Booking:** The system successfully blocks any reservation that overlaps with an existing booking's date range.
- **Data Validation:** Using **Encapsulation**, the system rejects negative balances and empty usernames, throwing custom exceptions.
- **Date Logic:** The system prevents "check-out" dates from being set before "check-in" dates.
- **Role Security:** Verified that **Guest** users are blocked from accessing **Admin** management functions via **Enum** role checks.

5.0 Limitations

- **Data is not saved permanently:** Because the system uses an "In-Memory Database," all new information like registered guests or new bookings is lost as soon as the program is closed. We haven't added a way to save data to a file yet.
- **No visual interface:** For this first milestone, the system only works through the console. Users have to type their commands and read text outputs instead of clicking on buttons or using a window.

- **Simple search engine:** The search feature only works if you type the room type or amenities exactly right. It cannot find a room if there is a typo, and it doesn't have a way to sort rooms by price yet.
- **Works on one computer only:** This is a local application, so it cannot be used by different people on different computers at the same time. Everything happens on one machine.
- **Fake payment system:** The payment feature is just a simulation. It doesn't connect to a real bank; it only checks if a number in the user's account balance is high enough to cover the cost.

Appendix A: UML Class Diagram